

# 课程学习小组讨论记录

【讨论主题】：项目需求分析及设计方案——星露谷战斗模块

【课程名称】：软件项目开发与实践

【讨论日期】：2026 年 6 月

【讨论时长】：2.5 小时

【主持人】：谢宇鹏

【参与人员】：谢宇鹏（学号：5120243390）

## 第 1 部分 项目背景与目标

### 【项目背景】

近年来，独立游戏市场蓬勃发展，像素风俯视角动作 RPG 因其经典的游戏体验和较低的开发门槛，成为游戏开发教学的理想载体。《星露谷物语》（Stardew Valley）是一款以农场经营为核心的模拟 RPG，其矿洞战斗系统深受玩家喜爱——玩家在俯视角像素风画面中，使用多种武器与怪物实时战斗，收集资源、提升能力。然而，《星露谷物语》的核心玩法侧重于农场经营和社交，战斗系统只是其中一个子模块。

与此同时，《塞尔达传说》（The Legend of Zelda）系列作为俯视角动作 RPG 的鼻祖，其战斗系统设计精良——多武器切换、实时攻击判定、怪物 AI 行为模式，为本项目提供了重要的设计参考。

当前市场上缺少一款以战斗和探索为核心玩法的独立小游戏，能够将《星露谷物语》的像素美术风格与《塞尔达传说》的战斗探索机制融合。本项目正是基于这一需求，提炼两者的核心元素，打造一款名为「星露谷战斗模块」的俯视角像素风动作 RPG。

### 【项目目标】

功能性目标：

1. 实现俯视角像素风画面，支持多图层瓦片地图渲染和 Y 轴排序遮挡关系
2. 实现玩家角色的四方向移动、待机、攻击共 12 组动画（4 方向 × 3 状态）
3. 实现近战攻击系统，支持五种武器（剑、长矛、斧头、刺剑、叉）的实时切换
4. 实现魔法系统，包含治疗（回复生命值）和火焰（前方范围伤害）两种魔法

5. 实现四种怪物的 AI 系统，具备待机、追踪、攻击三种行为状态，基于有限状态机
6. 实现 A\* 寻路算法，敌人可绕过障碍物追踪玩家
7. 实现多张地图（至少 4 张）的场景切换，带渐入渐出过渡效果
8. 实现角色升级系统，通过击杀怪物获得经验值，可分配到五项属性
9. 实现 JSON 格式的手动存档与读档功能
10. 实现粒子特效系统（攻击命中、死亡、治疗、草地碎片等）

#### 非功能性目标：

- 游戏帧率稳定在 60FPS 以上，窗口分辨率 1280 × 720
- 采用面向对象架构，多文件多模块结构，符合软件工程规范
- 所有资源引用使用相对路径，确保项目可移植性
- 资源文件缺失时静默降级，游戏不崩溃
- 游戏启动后 3 秒内显示主菜单

## 第 2 部分 主要讨论内容

### 功能需求分析

经过详细讨论，将功能需求按优先级分为三级，共 14 个功能点：

#### 核心功能（P0）——必须实现

| 编号    | 功能点   | 详细说明                                                                      | 优先级 |
|-------|-------|---------------------------------------------------------------------------|-----|
| FR-01 | 主菜单场景 | 深色背景 + 浮动粒子特效，游戏标题带正弦浮动动画，菜单选项「开始游戏」「退出游戏」支持上下键选择和高亮脉冲效果，播放背景音乐，支持 ESC 退出 | P0  |

| 编号    | 功能点      | 详细说明                                                                                           | 优先级 |
|-------|----------|------------------------------------------------------------------------------------------------|-----|
| FR-02 | 玩家移动与动画  | 方向键/WASD 控制四方向移动, 角色根据移动方向切换朝向和动画帧 (移动/待机/攻击共 12 组), 支持对角线移动但速度归一化防止加速, 分轴 AABB 碰撞检测防止穿墙       | P0  |
| FR-03 | 近战攻击系统   | 空格键触发攻击, 根据玩家朝向在对应方向生成武器精灵, 武器精灵与敌人发生碰撞时扣血, 攻击有冷却时间 (不同武器冷却不同), 播放攻击音效                         | P0  |
| FR-04 | 魔法系统     | 左 Ctrl 触发当前选中魔法, 治疗魔法回复生命值并生成光环/治疗粒子特效, 火焰魔法在朝向前方释放多个火焰粒子覆盖范围伤害, 两种魔法均消耗能量值                    | P0  |
| FR-05 | 武器切换     | Q 键在五种武器间循环切换 (剑/长矛/斧头/刺剑/叉), 每种武器有不同的伤害值和冷却时间, 切换时有短暂冷却防止误操作                                  | P0  |
| FR-06 | 敌人 AI 系统 | 四种怪物 (鱿鱼/浣熊/精灵/竹子) 各有不同属性, AI 采用有限状态机 (待机/追踪/攻击), 追踪时使用 A* 寻路绕过障碍物, 被攻击后短暂无敌并闪烁, 死亡时播放特效并掉落经验值 | P0  |
| FR-07 | 地图系统     | CSV 格式瓦片地图, 由四个图层叠加 (碰撞层/草地层/装饰层/实体层), 视窗小于完整地图并跟随玩家移动, 地图边界检测防止越界                             | P0  |
| FR-08 | 场景切换     | 地图上设有传送点, 玩家触碰后触发切换, 先淡出 (画面变黑) 再加载新地图再淡入, 传送过程中玩家属性/装备/经验跨地图保留, 传送点有防抖机制                      | P0  |

重要功能 (P1) —— 应该实现

| 编号    | 功能点    | 详细说明                                                                                      | 优先级 |
|-------|--------|-------------------------------------------------------------------------------------------|-----|
| FR-09 | 升级系统   | 按 M 键暂停游戏并打开升级菜单，菜单显示当前五项属性（生命/能量/攻击/魔法/速度）和升级消耗，玩家选择属性后消耗经验值提升，每项属性有上限                   | P1  |
| FR-10 | 存档系统   | JSON 格式存档，暂停菜单中按 P 键保存，存档内容包括玩家位置/血量/能量/经验/属性/装备选择以及已击败敌人坐标和已破坏草地坐标，启动时检测存档并弹出对话框选择继续或新游戏 | P1  |
| FR-11 | 粒子特效   | 攻击命中时产生火花特效，敌人死亡时产生对应怪物类型的死亡特效，治疗时产生光环和治疗粒子，草地被破坏时产生叶子碎片特效，经验值飞向玩家时产生轨迹特效                 | P1  |
| FR-12 | HUD 显示 | 屏幕上方持续显示：红色血条（实时更新）、蓝色能量条（实时更新）、当前武器图标、当前魔法图标、经验值数值                                       | P1  |

### 辅助功能（P2）——可以实现

| 编号    | 功能点  | 详细说明                                                                            | 优先级 |
|-------|------|---------------------------------------------------------------------------------|-----|
| FR-13 | 死亡界面 | 玩家 HP 降至 0 时显示，暗红色背景，大字“You Died”，显示本局获得经验值，提示按 R 重新开始/按 ESC 退出，红色浮动粒子特效，播放死亡音效 | P2  |
| FR-14 | 音效系统 | 主菜单背景音乐循环播放，攻击音效（剑击/魔法释放），受伤音效，死亡音效，菜单选择音效，升级音效，所有音效支持音量调节                      | P2  |

## 场景设计

### 场景一：主菜单

主菜单是玩家进入游戏的第一个界面，设计目标是简洁美观、操作直觉。

- **背景：**深蓝色渐变背景（RGB: 20, 20, 40），营造神秘氛围
- **粒子效果：**蓝色浮动粒子缓慢上升，增加动态感
- **标题：**金色大字”星露谷战斗模块”，带正弦函数浮动动画，增加灵动感
- **副标题：**灰色小字说明游戏类型
- **菜单项：**「开始游戏」和「退出游戏」，选中项有脉冲高亮效果（每 100ms 闪烁）
- **操作提示：**底部显示操作说明
- **音频：**播放主菜单背景音乐，循环播放

### 场景二：游戏关卡

游戏关卡是核心场景，包含以下子系统：

- **地图渲染：**地面贴图作为底层，四层 CSV 数据叠加渲染
- **视窗系统：**屏幕窗口  $1280 \times 720$ ，小于完整地图，以玩家为中心截取地图局部区域
- **玩家角色：**像素精灵，12 组动画（上/下/左/右  $\times$  移动/待机/攻击）
- **敌人系统：**四种怪物在地图上按预设位置生成，AI 驱动行为
- **可破坏物：**草地精灵可被攻击摧毁，产生叶子碎片粒子
- **障碍物：**树木、石墙等不可通行区域
- **传送点：**地图上特定位置，触碰后切换地图
- **HUD：**屏幕顶部固定显示血条/能量条/武器/经验
- **升级菜单：**暂停后覆盖显示，半透明背景

### 场景三：死亡界面

- 背景：暗红色（RGB: 40, 0, 0），营造死亡氛围
- 标题：红色大字”You Died”
- 统计：显示本局获得的经验值
- 选项：「按 R 重新开始」「按 ESC 退出」
- 粒子：红色浮动粒子特效
- 音效：播放死亡音效

### 场景四：存档对话框

- 触发：启动游戏时检测到存档文件
- 外观：圆角矩形对话框，深色背景，金色边框，带阴影效果
- 内容：标题”检测到存档！”，提示”按 C 继续 / 按 N 新游戏”
- 按钮：[C] Continue（绿色）/ [N] New Game（红色）

## 角色与装备

### 玩家属性设计

| 属性  | 初始值 / 上限  | 设计理由                          |
|-----|-----------|-------------------------------|
| 生命值 | 200 / 600 | 初始 200 保证新手不易死亡，上限 600 提供成长空间 |
| 能量值 | 100 / 300 | 控制魔法释放频率，防止无脑放技能              |
| 攻击力 | 15 / 30   | 初始 15 配合武器伤害，3 刀左右杀一只入门怪      |
| 魔法  | 5 / 15    | 影响魔法伤害和能量回复速度，低数值高影响          |
| 速度  | 300 / 720 | 300 像素/秒为基础速度，720 为最大速度       |

武器系统设计

| 武器 | 伤害 | 冷却        | 设计定位                 |
|----|----|-----------|----------------------|
| 剑  | 15 | 短 (100ms) | 均衡型，新手友好，适合作为默认武器    |
| 长矛 | 30 | 长 (400ms) | 高伤慢速，适合精准打击，一击制敌     |
| 斧头 | 20 | 中 (300ms) | 中等性能，过渡武器，适合中期使用     |
| 刺剑 | 8  | 极短 (50ms) | 极快攻速，连击流，DPS 最高但需要近身 |
| 叉  | 10 | 短 (80ms)  | 快速灵活，适合走位流打法         |

伤害计算公式：实际伤害 = 角色攻击力 + 武器伤害

魔法系统设计

| 魔法 | 效果      | 设计细节                                   |
|----|---------|----------------------------------------|
| 治疗 | 回复 50HP | 消耗 10 能量，生成光环 + 治疗粒子特效，血量不超过最大值        |
| 火焰 | 前方范围伤害  | 消耗 20 能量，在朝向方向 5 格内生成 5 个火焰粒子，随机偏移覆盖范围 |

敌人系统设计

| 怪物 | 血量      | 伤害     | 经验      | 攻击方式 | AI 特点          |
|----|---------|--------|---------|------|----------------|
| 竹子 | 极低 (35) | 极低 (5) | 中 (120) | 叶子攻击 | 入门级，速度适中，警戒范围小 |
| 鱿鱼 | 低 (50)  | 中 (15) | 中 (100) | 近身斩击 | 速度适中，中等警戒范围    |
| 精灵 | 低 (40)  | 低 (8)  | 中 (110) | 远程雷击 | 速度最快，低伤害       |
| 浣熊 | 高 (120) | 高 (25) | 高 (250) | 近身爪击 | 血厚伤害高，警戒范围最大   |

AI 行为规则详述：

1. 待机：玩家距离 > 警戒范围时原地不动，无路径计算开销

2. **追踪**: 玩家进入警戒范围后启用 A\* 寻路, 每 500ms 或玩家移动到新网格时重算路径
3. **攻击**: 玩家进入攻击范围后发动攻击, 播放攻击音效, 有 400ms 攻击冷却
4. **受击**: 被攻击后 300ms 无敌, 角色 alpha 值闪烁提示
5. **死亡**:  $HP \leq 0$  时播放死亡特效, 增加玩家经验值, 触发经验粒子飞向玩家

## 技术方案设计

| 层级   | 技术选型         | 选型理由                                      |
|------|--------------|-------------------------------------------|
| 编程语言 | Python 3.10+ | 语法简洁, 适合快速原型开发, 课程技术栈要求                   |
| 游戏引擎 | Pygame 2.x   | 跨平台 2D 引擎, 提供渲染/输入/音频/碰撞, 社区活跃            |
| 地图数据 | CSV 格式       | 比 TMX 更轻量, 无需额外解析库, Python 原生支持           |
| 寻路算法 | 待定           | 详见下文算法对比分析, 候选: A*/BFS/Dijkstra/贪心/DP/TSP |
| 架构模式 | 面向对象 (OOP)   | 多文件多模块, 类继承 + 组合, 符合课程要求                  |
| 存档格式 | JSON         | Python 内置 json 模块, 人类可读, 易于调试             |
| 版本管理 | Git          | 代码版本控制, 支持迭代回溯, B 模式要求                    |

## 寻路算法对比分析

敌人追踪玩家需要网格地图上找到一条从敌人位置到玩家位置的路径, 且该路径需要绕过障碍物。这是一个经典的图搜索问题, 我们对比了以下六种候选算法:

### 候选一: 广度优先搜索 (BFS)

- **原理**: 从起点开始逐层扩展, 先访问所有距离为 1 的节点, 再访问距离为 2 的节点, 直到找到终点



- 优点：一定能找到最短路径，实现简单，无需设计启发函数
- 缺点：无方向性，盲目扩展所有方向（上/下/左/右/对角线），搜索范围大
- 时间复杂度： $O(V + E)$ ， $V$  为节点数， $E$  为边数
- 适用场景：无权图的最短路径
- 评价：实现最简单，但搜索效率低，大量节点被无意义扩展

### 候选二：Dijkstra 算法

- 原理：从起点开始，每次选择当前累计代价最小的节点扩展，逐步到达终点
- 优点：保证最短路径，支持加权图（不同地形移动代价不同）
- 缺点：无启发式引导，搜索范围仍较大，向所有方向均匀扩展
- 时间复杂度： $O((V + E) \log V)$ ，使用优先队列
- 适用场景：加权图最短路径
- 评价：比 BFS 更通用，但本项目网格权重相同，优势不明显

### 候选三：A\* 算法

- 原理：综合 Dijkstra 和贪心搜索，使用评估函数  $f(n) = g(n) + h(n)$  引导搜索方向，其中  $g(n)$  为起点到当前节点的实际代价， $h(n)$  为当前节点到终点的启发式估计
- 启发函数：可选用曼哈顿距离  $h = |x_1 - x_2| + |y_1 - y_2|$  或欧几里得距离  $h = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- 优点：保证最短路径（启发函数满足一致性条件时），启发式引导大幅减少搜索范围
- 缺点：需要设计合适的启发函数，最坏情况退化为 Dijkstra
- 时间复杂度：最坏  $O(E)$ ，实际远小于此
- 评价：理论上最适合本场景——网格地图、静态障碍物、需最短路径

### 候选四：贪心最佳优先搜索

- 原理：每次只考虑启发函数  $h(n)$ ，选择离终点最近的节点扩展，不考虑已走过的代价
- 优点：搜索速度极快，直奔目标方向

- **缺点：**不保证最短路径，可能走入死胡同后需要回溯
- **时间复杂度：** $O(E)$
- **评价：**速度最快但路径质量无保证，若障碍物布局复杂可能找到很绕的路径

#### 候选五：动态规划（DP）方法

- **原理：**将网格地图视为动态规划表，从终点反向计算每个格子到终点的最短距离，敌人只需朝距离值减小的方向移动
- **优点：**一次计算全局最优，后续查询  $O(1)$ ；适合敌人数量多的场景
- **缺点：**地图有动态变化时（如可破坏障碍物）需要重新计算全表；内存开销较大
- **时间复杂度：**预计算  $O(V)$ ，查询  $O(1)$
- **评价：**适合静态地图，但本项目有可破坏的草地障碍物，地图会变化，预计算失效

#### 候选六：旅行商问题（TSP）方法

- **原理：**将多个敌人视为旅行商，需要规划每个敌人访问玩家的最短路线，同时考虑多敌人之间的路径不冲突
- **优点：**理论上可以得到多敌人的全局最优路径规划
- **缺点：**TSP 是 NP-hard 问题，精确求解的时间复杂度为  $O(n!)$ ，即使使用近似算法也代价高昂
- **时间复杂度：**精确解  $O(n!)$ ，近似解  $O(n^2 2^n)$ （动态规划 bitmask）
- **评价：**理论意义大于实际意义。本项目敌人独立行动，不需要协调多敌人路径，TSP 的复杂度远超需求

#### 综合对比

##### 初步结论

经过对比分析，**A\* 算法**在本项目场景下表现最优：既保证最短路径，又通过启发式引导大幅提高搜索效率，且支持动态地图（草地被破坏后路径自动更新）。贪心搜索虽然最快但路径质量不可控，动态规划在地图变化时需要重新计算，TSP 的复杂度远超本项目需求。

**最终选型建议：**推荐采用 A\* 算法，使用曼哈顿距离作为启发函数。但该结论需在编码实现阶段通过实际性能测试验证，若 A\* 在大量敌人同时寻路时出现性能瓶颈，可考虑降级为贪心搜索作为 fallback 方案。

| 算法        | 最短路径      | 搜索效率         | 实现难度 | 动态地图      | 评价          |
|-----------|-----------|--------------|------|-----------|-------------|
| BFS       | 保证        | 低            | 简单   | 支持        | 效率太低        |
| Dijkstra  | 保证        | 中            | 中等   | 支持        | 可用          |
| <b>A*</b> | <b>保证</b> | <b>高</b>     | 中等   | <b>支持</b> | <b>推荐</b>   |
| 贪心搜索      | 不保证       | 极高           | 简单   | 支持        | 路径质量差       |
| 动态规划      | 保证        | 极高 (预计<br>算) | 中等   | 不支持       | 地图变化时<br>失效 |
| TSP       | 全局最优      | 极低           | 极高   | 不需要       | 过度设计        |

## 核心业务流程

### 游戏主循环

1. 游戏启动 → 初始化 Pygame → 加载所有资源
2. 显示主菜单 → 等待玩家选择
3. 开始游戏 → 初始化关卡（解析地图、创建精灵）
4. 每帧循环：处理输入事件 → 更新游戏逻辑 → 检测碰撞 → 渲染画面 → 刷新显示
5. 检测特殊事件（死亡/传送）→ 切换状态

### 战斗交互流程

1. 玩家按下攻击键 → 角色进入攻击状态
2. 关卡层在角色朝向方向创建武器精灵
3. 武器精灵根据朝向放置在对应方位（上/下/左/右）
4. 每帧检测武器精灵与所有敌人的碰撞
5. 碰撞发生 → 敌人扣除伤害 → 播放受击特效和音效
6. 攻击冷却结束 → 移除武器精灵 → 角色恢复可操作状态

## 场景切换流程

1. 每帧检测玩家是否与传送点坐标重叠（距离 < 半格）
2. 触发传送 → 执行淡出动画（alpha 从 0 递增到 255）
3. 销毁当前关卡的所有精灵和数据
4. 根据传送点配置加载目标地图，创建新关卡实例
5. 将玩家放置到目标地图的指定出生点
6. 执行淡入动画（alpha 从 255 递减到 0）
7. 玩家属性/装备/经验跨地图保留

## 主要难点

| 难点    | 说明           | 解决思路               |
|-------|--------------|--------------------|
| 分轴碰撞  | 同时处理两轴可能对角穿墙 | 先水平后垂直分两步修正位置      |
| A* 性能 | 多敌人同时寻路可能掉帧  | 路径缓存 + 定时重算，避免每帧计算 |
| 视窗越界  | 地图大于视窗时的边界处理 | 计算偏移量并限制在地图范围内     |
| 状态保持  | 场景切换时保留玩家数据  | 玩家对象跨关卡传递，只更新引用    |
| Y 轴遮挡 | 精灵间正确遮挡关系    | 每帧按 Y 坐标排序后依次绘制    |
| 动画同步  | 多组动画帧数不同     | 帧索引按浮点累加，超出时取模归零   |

## 第3部分 总结与计划

### 讨论总结

本次讨论全面分析了「星露谷战斗模块」项目的需求和技术方案。项目定位为一款以战斗和探索为核心的俯视角像素风动作 RPG，核心玩法包括多武器近战战斗、双魔法系统、四态敌人 AI、多地图场景切换和角色升级系统。

技术上采用 Python + Pygame 的轻量级方案，面向对象架构，CSV 地图数据，JSON 存档，A\* 寻路算法。经过 BFS、Dijkstra、A\*、贪心搜索四种算法的对比分析，最终确定使用 A\* 算法作为寻路方案。

### 后续开发计划

1. 第 1 天：搭建项目框架，实现游戏主循环、主菜单场景、基础精灵类
2. 第 2 天：实现地图 CSV 解析系统、四层地图渲染、视窗跟随
3. 第 3 天：实现玩家移动/动画、碰撞检测、近战攻击和魔法系统
4. 第 4 天：实现敌人 AI 状态机、A\* 寻路算法、敌人行为
5. 第 5 天：实现多地图场景切换、传送点系统、渐入渐出效果
6. 第 6 天：实现存档系统、升级系统、粒子特效、HUD 显示
7. 第 7 天：全面测试、Bug 修复、性能优化